Piyush Gupta

Roll No. 2401033

T12 - AI&DS

Sub : ASD&D Lab

EXPERIMENT – 8

AIM : To build an automated CI/CD pipeline on AWS that fetches source code from GitHub, builds it with AWS CodeBuild, and deploys it to an Amazon S3 static website using AWS CodePipeline.

LO Mapped : LO4,LO6

LO4 : Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

LO6 : Develop and demonstrate an end-to-end DevOps pipeline mini-project integrating all stages of DevOps from code commit to deployment and monitoring.

Theory :

CI/CD stands for Continuous Integration and Continuous Deployment. It is a software development practice that automates the process of integrating, building, and deploying application code, reducing manual effort and improving the speed and reliability of software delivery.

In this experiment, an automated CI/CD pipeline is implemented using AWS CodePipeline, AWS CodeBuild, GitHub, and Amazon S3. GitHub is used as the source code repository, AWS CodePipeline automates the workflow, and AWS CodeBuild builds the application using the

instructions specified in the buildspec.yml file. The generated website files are then deployed to an Amazon S3 bucket configured for static website hosting.

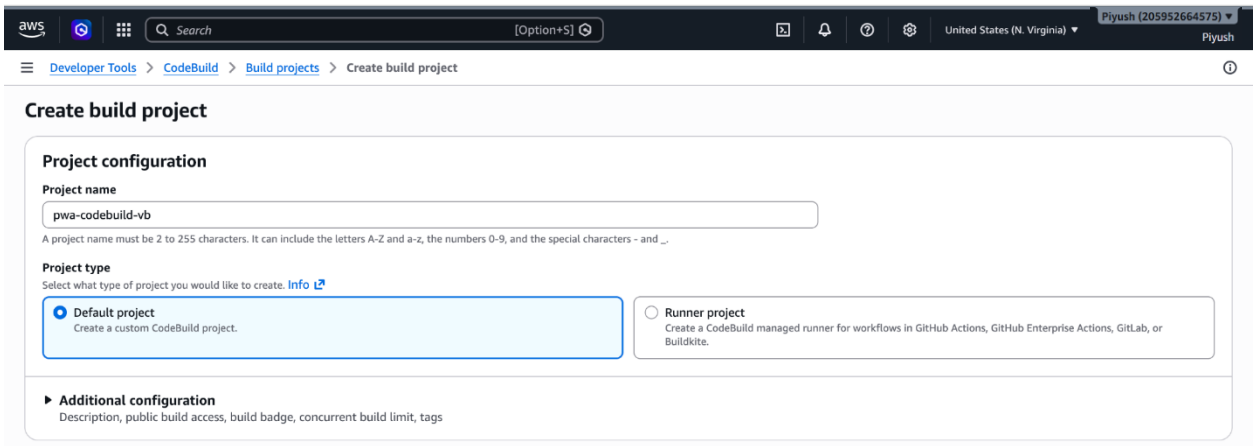
Whenever changes are pushed to the configured GitHub repository branch, AWS CodePipeline initiates the pipeline, retrieves the updated source code, and triggers the CodeBuild stage. After a successful build, the website files are deployed to the S3 bucket, where they can be accessed through the static website endpoint.

Thus, this pipeline demonstrates how AWS services can automate the complete process of fetching source code from GitHub, building the application, and deploying it to an Amazon S3 static website.

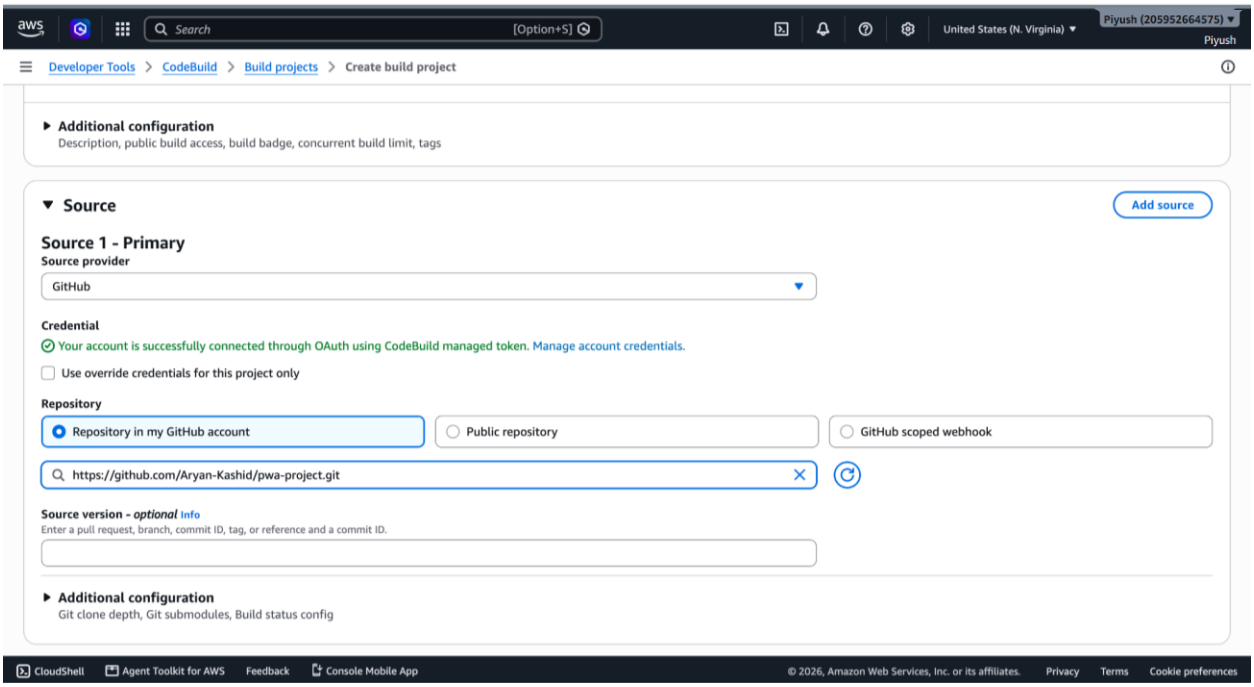
Execution :

1. Create a new build project in AWS CodeBuild named "pwa-codebuild-vb".

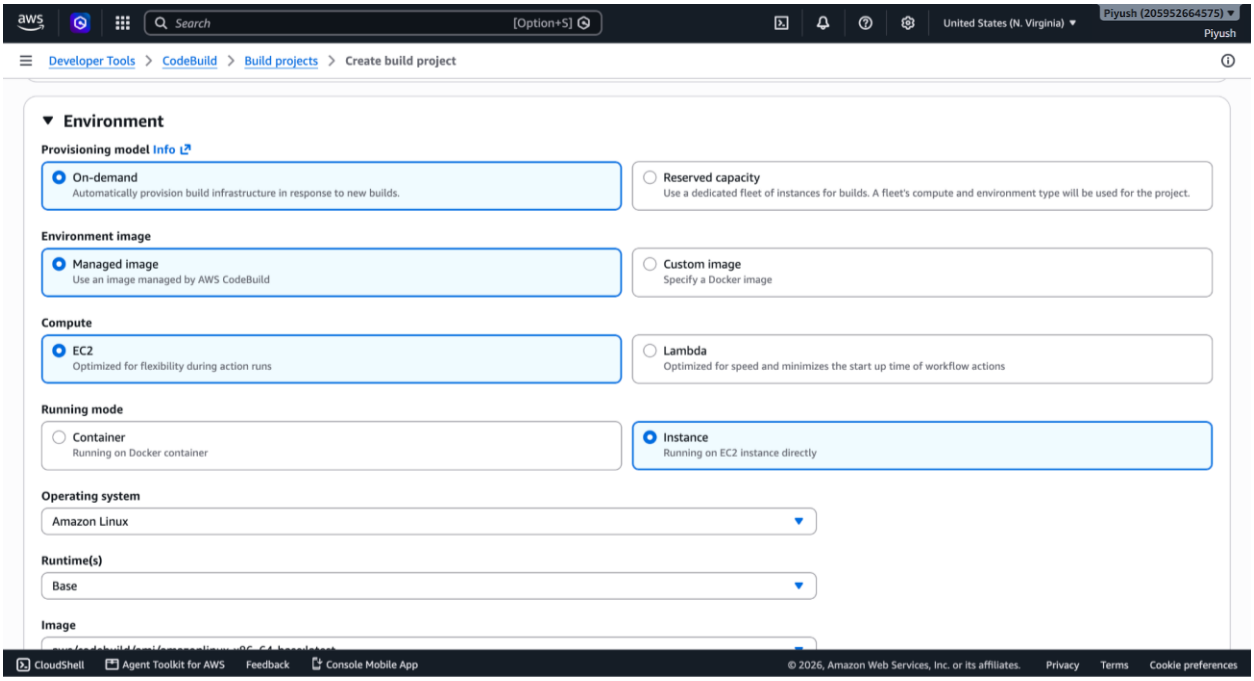
2. Configure the source provider as GitHub and select OAuth app credential type.



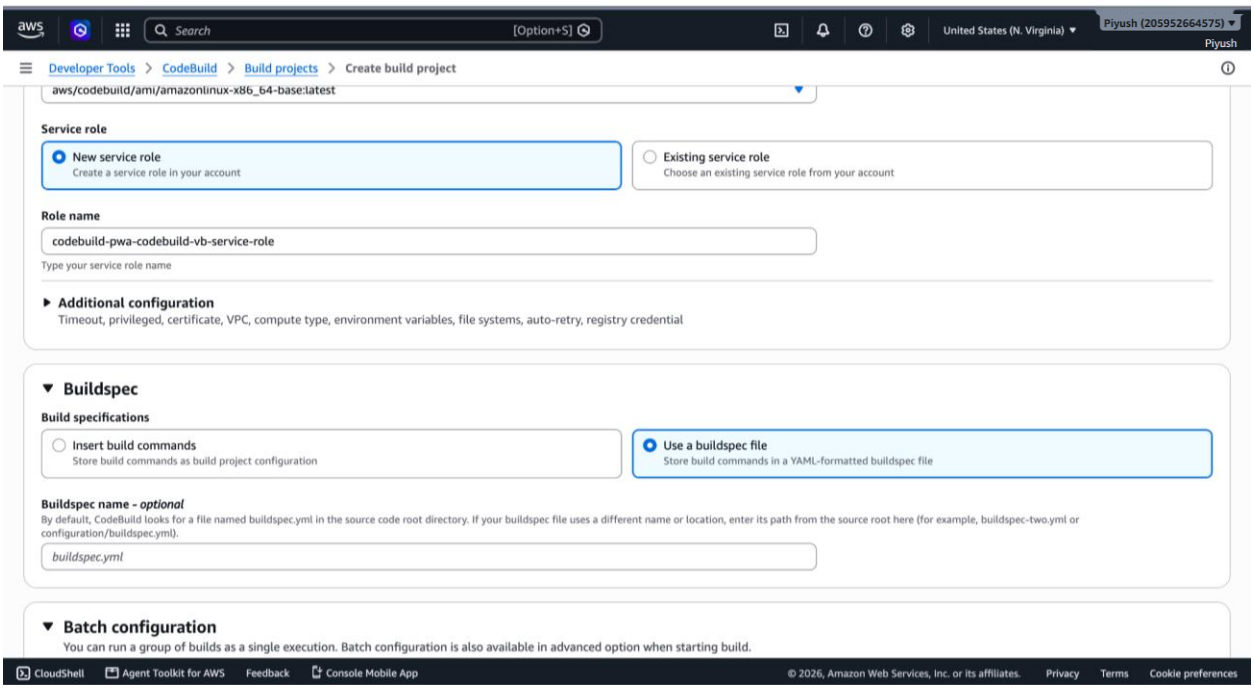
3. Authorize AWS CodeBuild to access the GitHub account and Connect the source repository URL to CodeBuild.



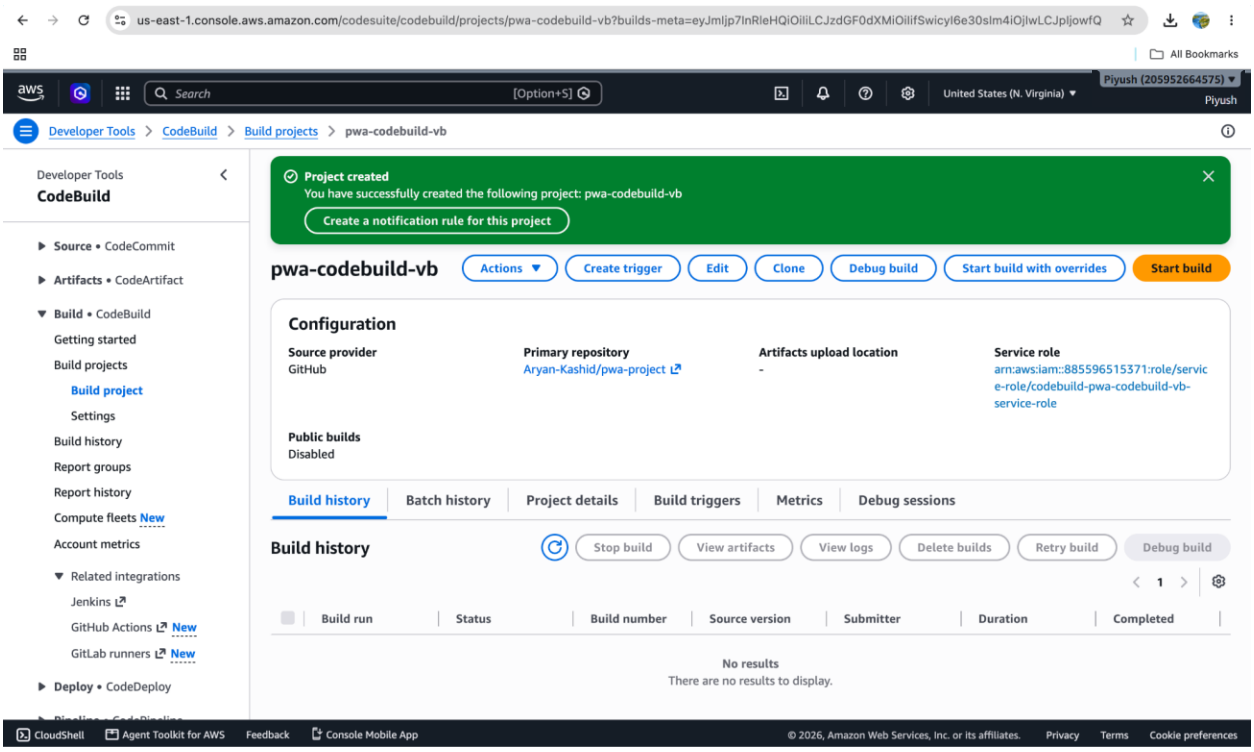
5. Configure the build environment settings, including On-demand provisioning, Managed image, and Amazon Linux OS.



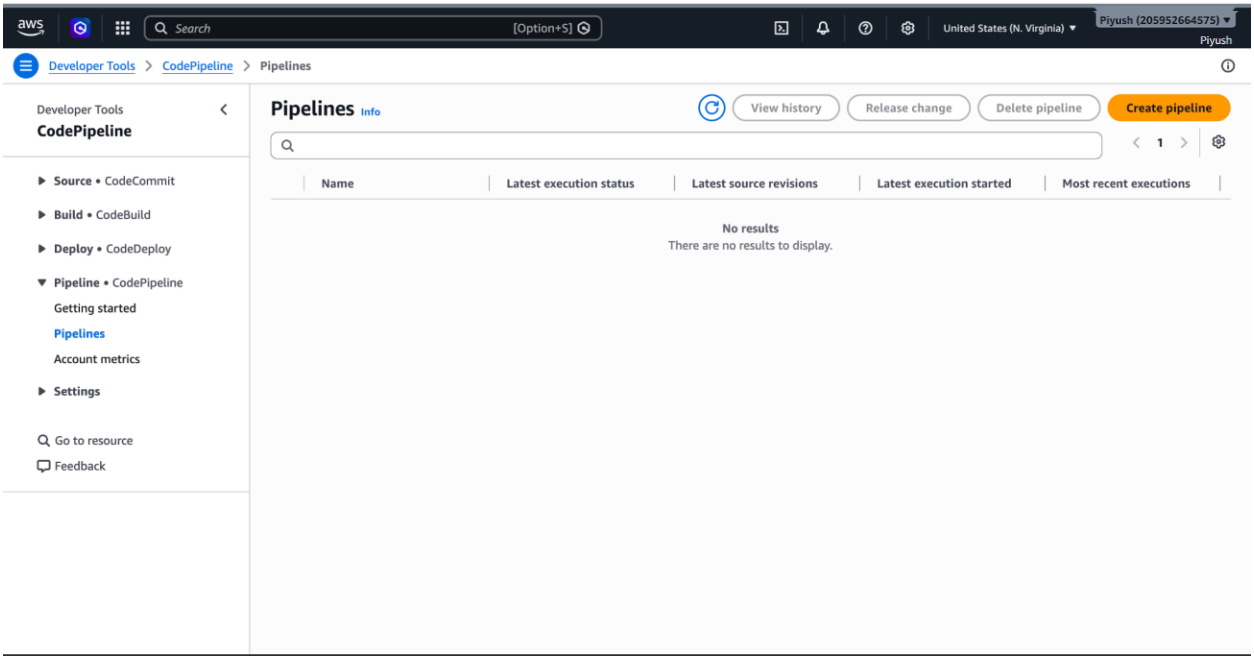
6. Set up the service role and specify the buildspec file configuration.

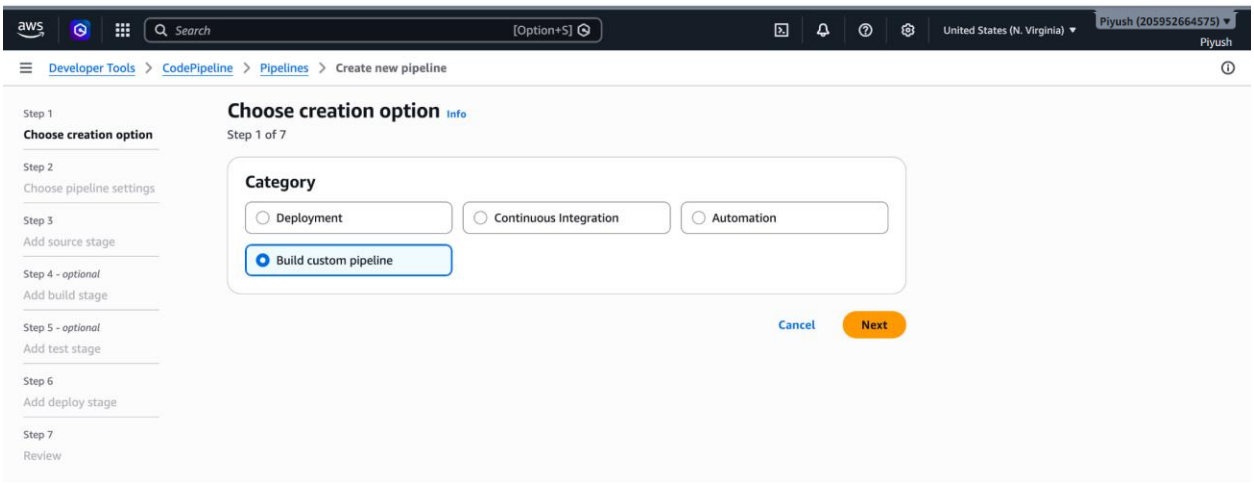


7. Verify that the CodeBuild project is successfully created.

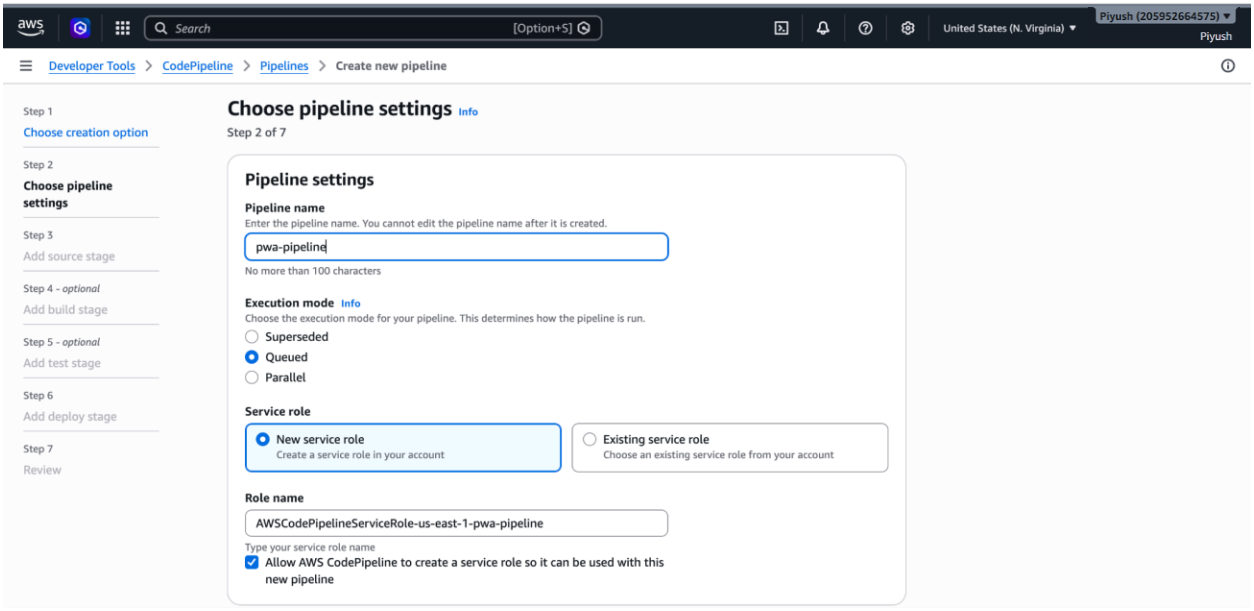


8. Navigate to AWS CodePipeline and choose the custom pipeline creation option.

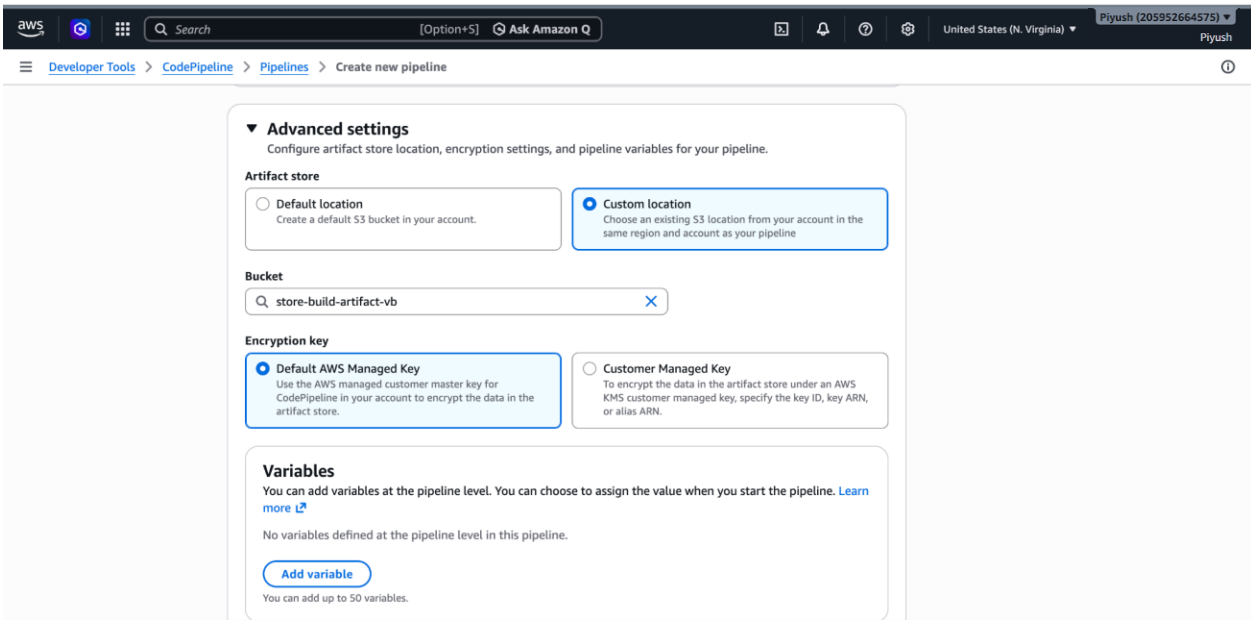




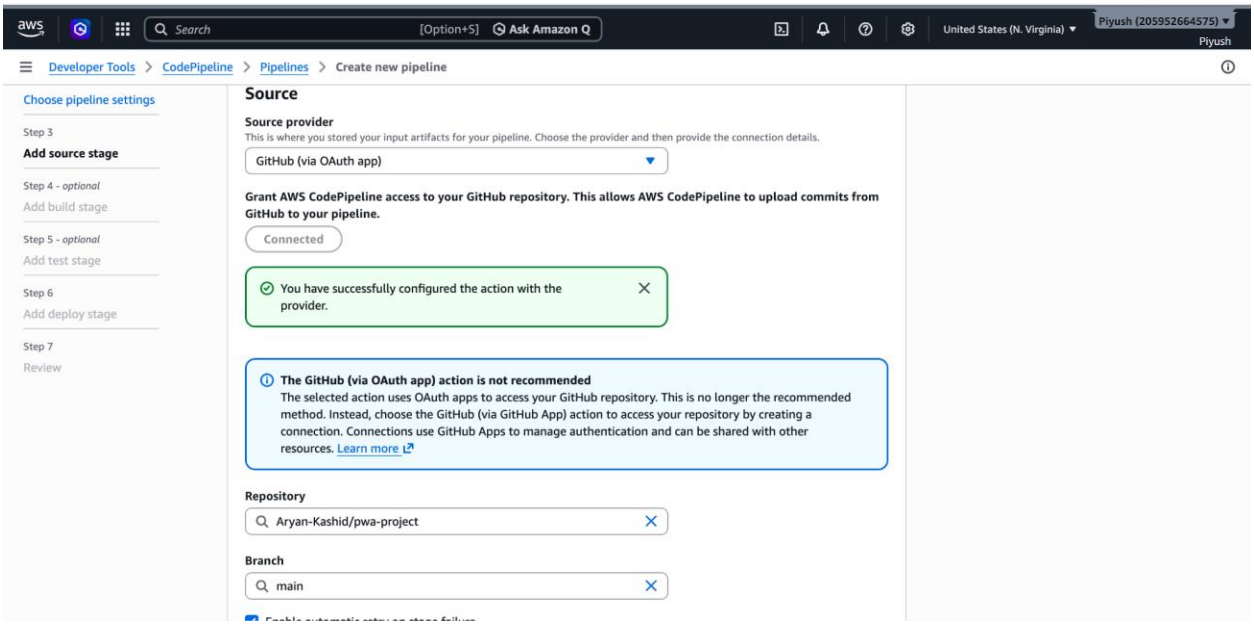
9. Enter the pipeline name ("pwa-pipeline") and service role details.



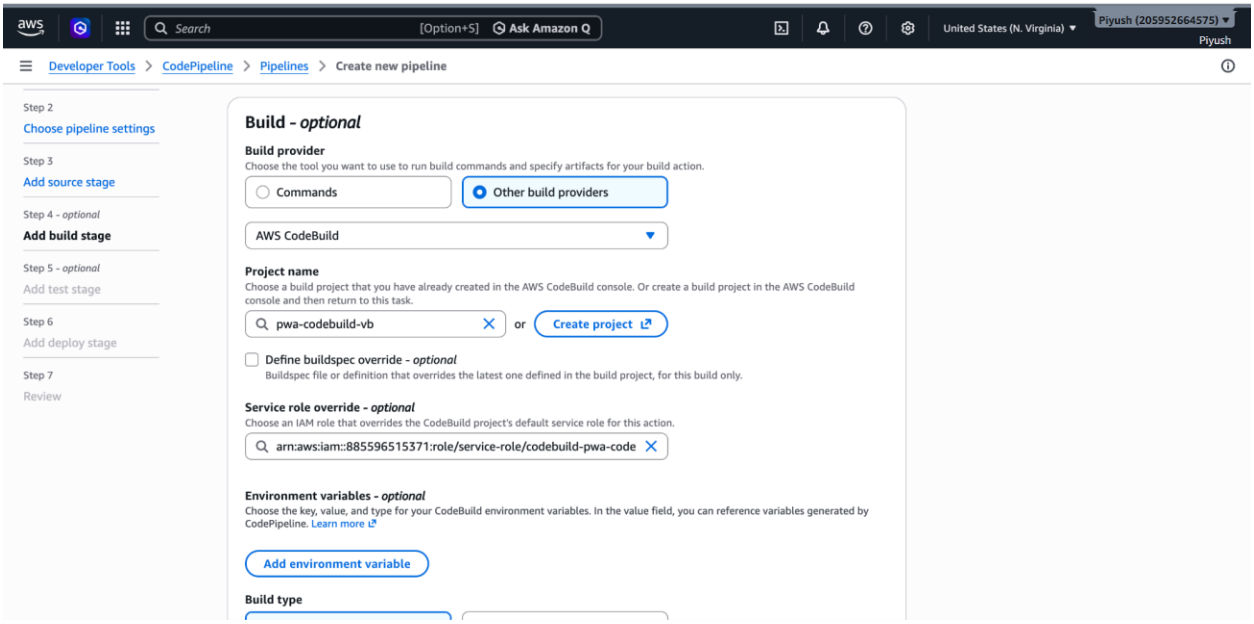
10. Set up advanced settings and specify the S3 artifact store location.



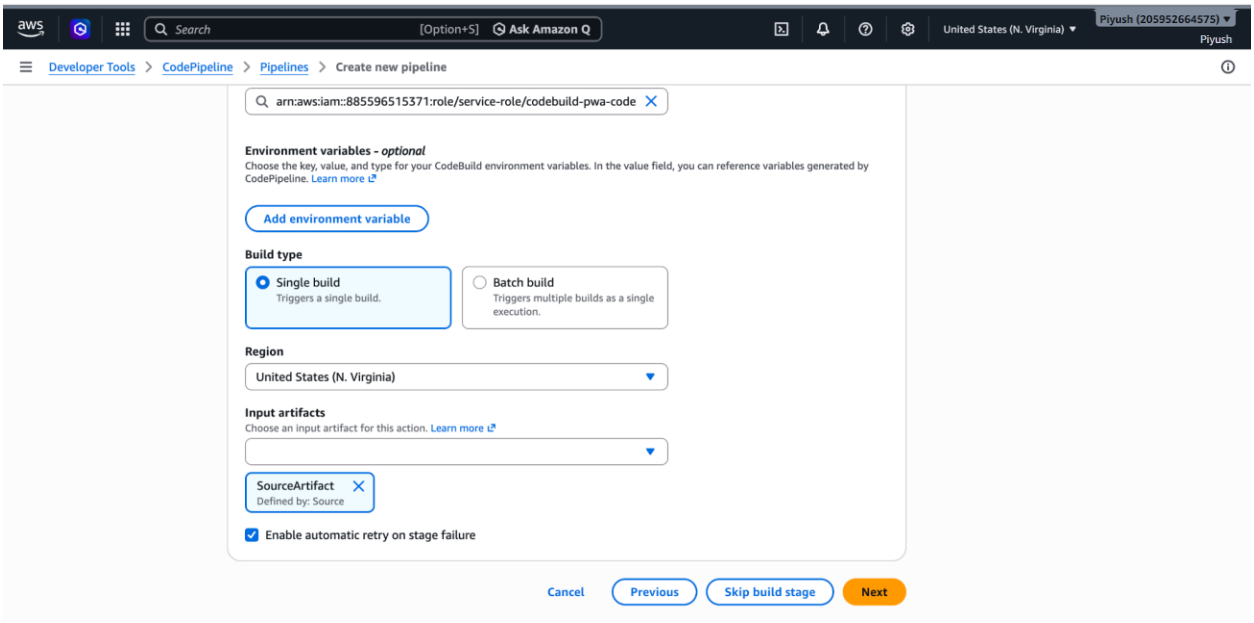
11. Add GitHub as the source stage provider and connect the target repository and branch.



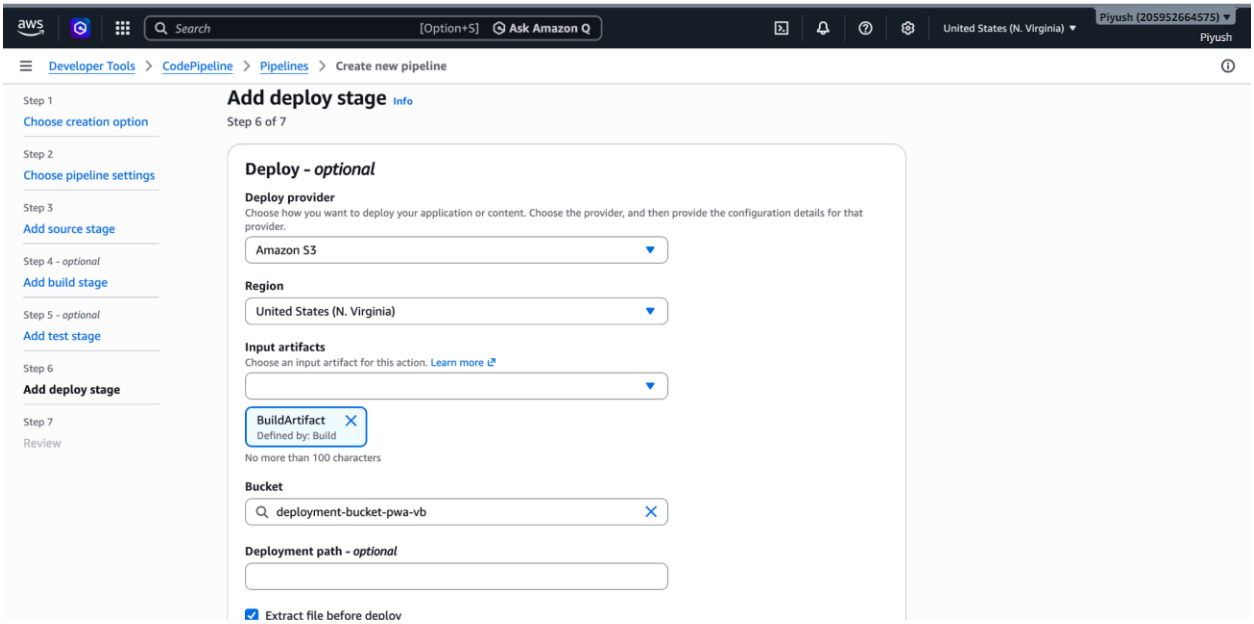
12. Configure the build stage using AWS CodeBuild and select the created project.



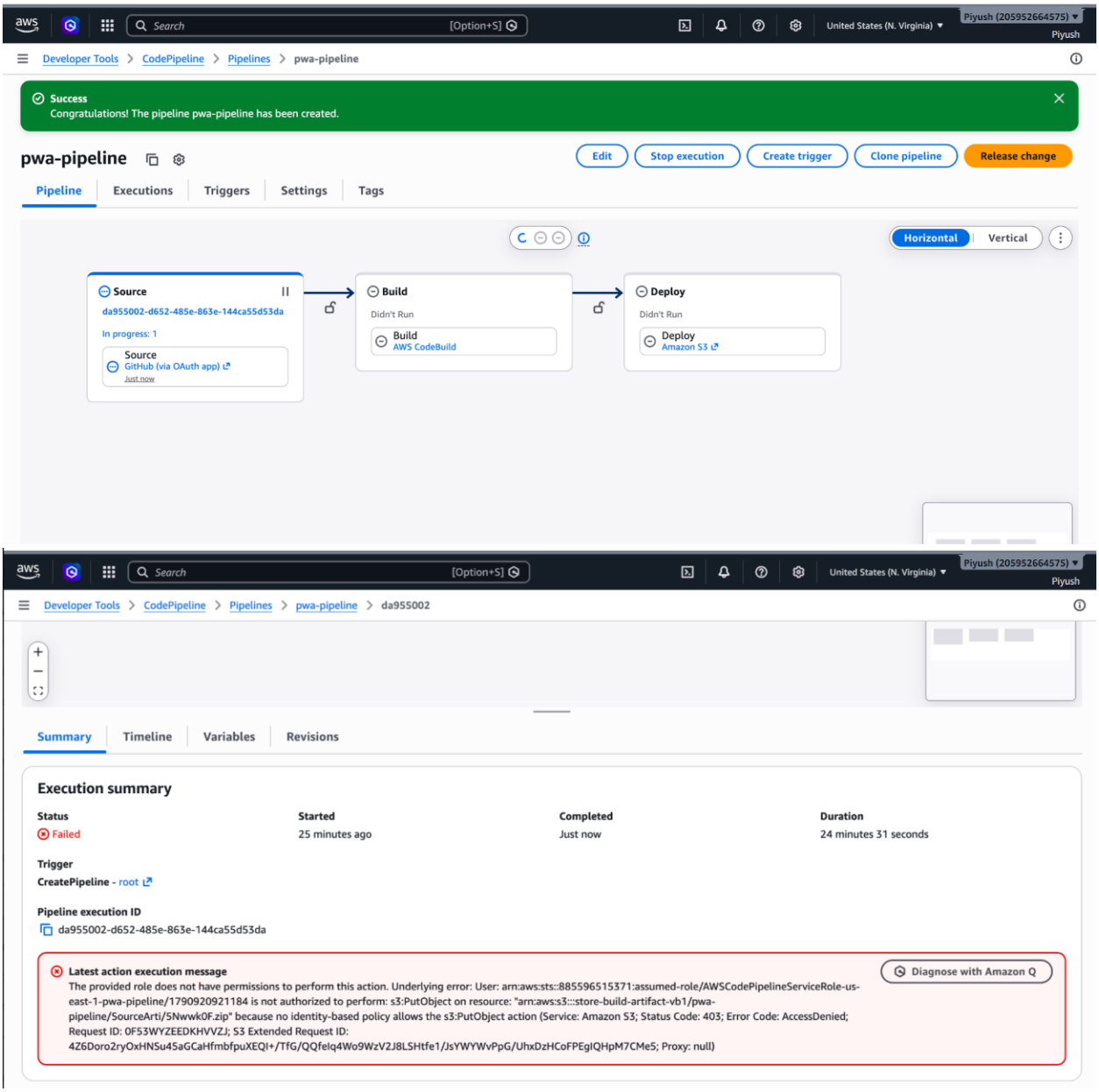
13. Specify single build execution and select input artifacts.



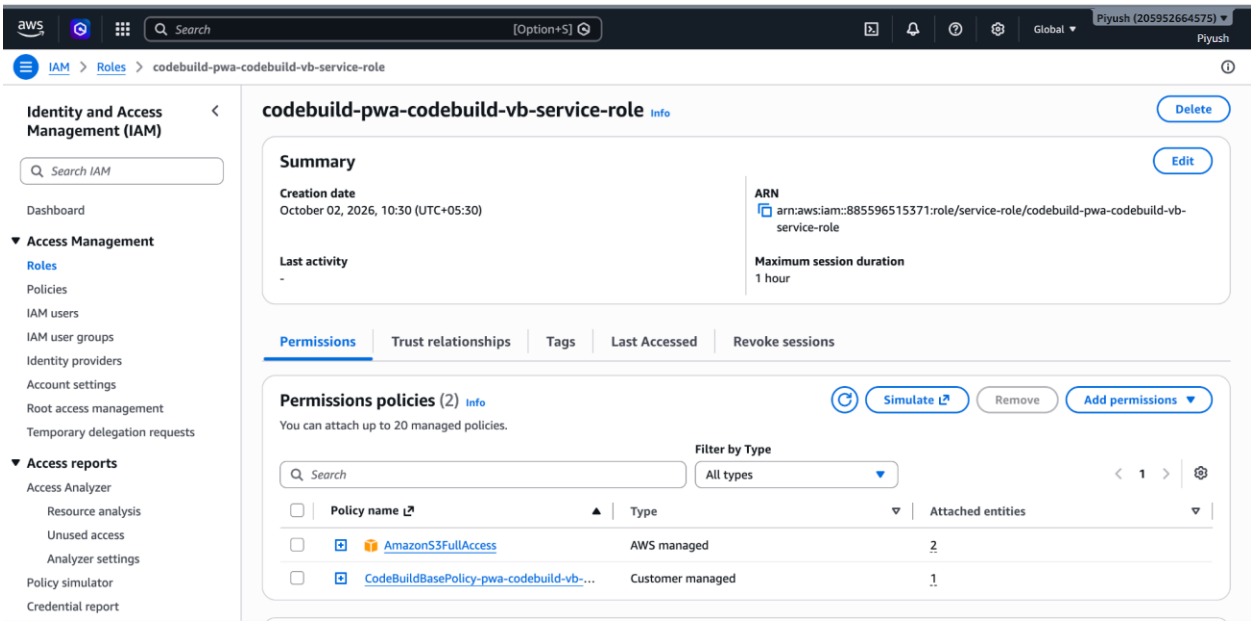
14. Configure Amazon S3 as the deployment provider and select the deployment bucket.



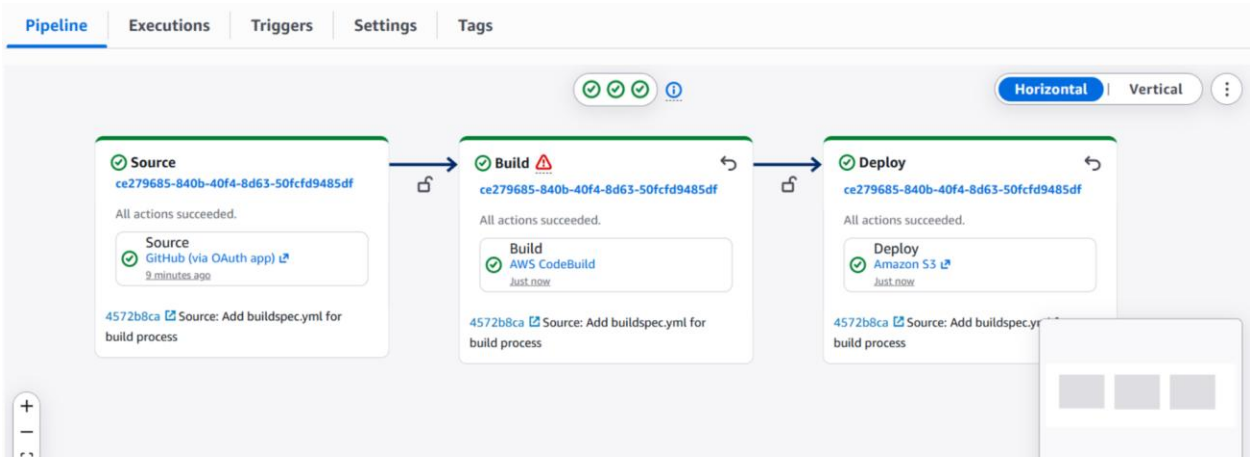
15. Review pipeline creation and identify any initial permission/execution errors.



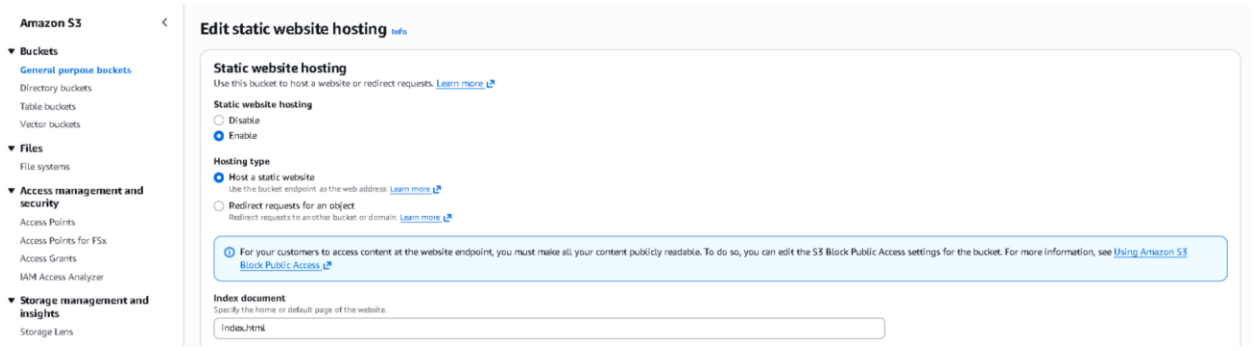
16. Attach necessary permissions policies (such as AmazonS3FullAccess) to the IAM service role.



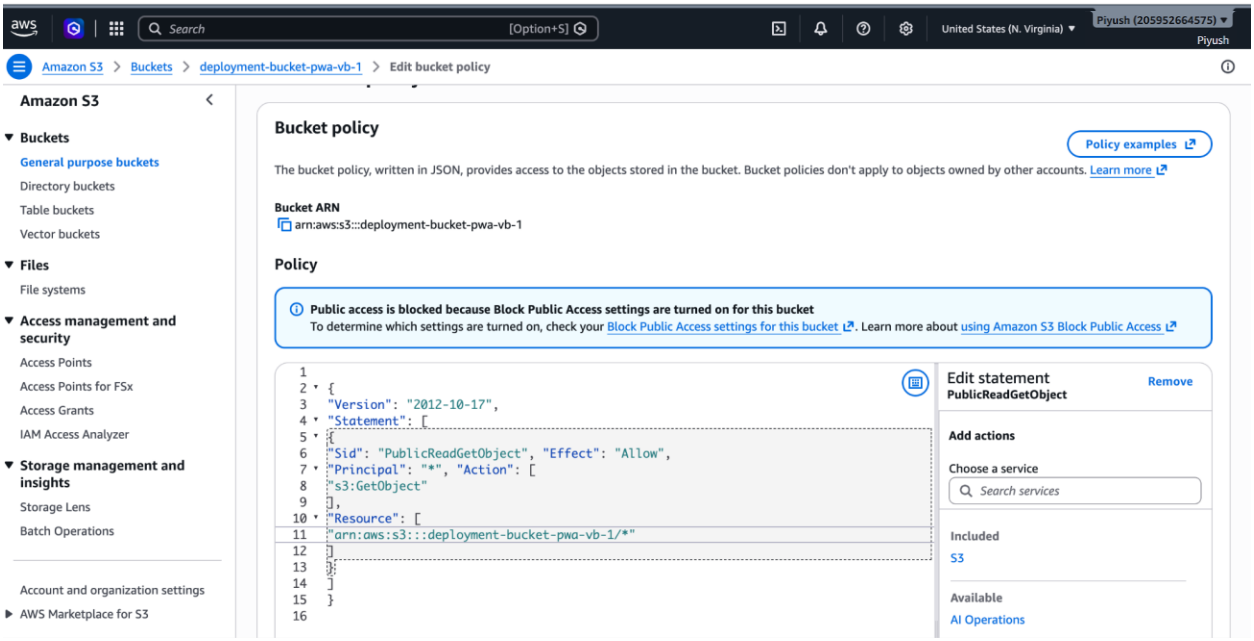
17. Verify successful execution across all stages (Source, Build, Deploy) in AWS CodePipeline.



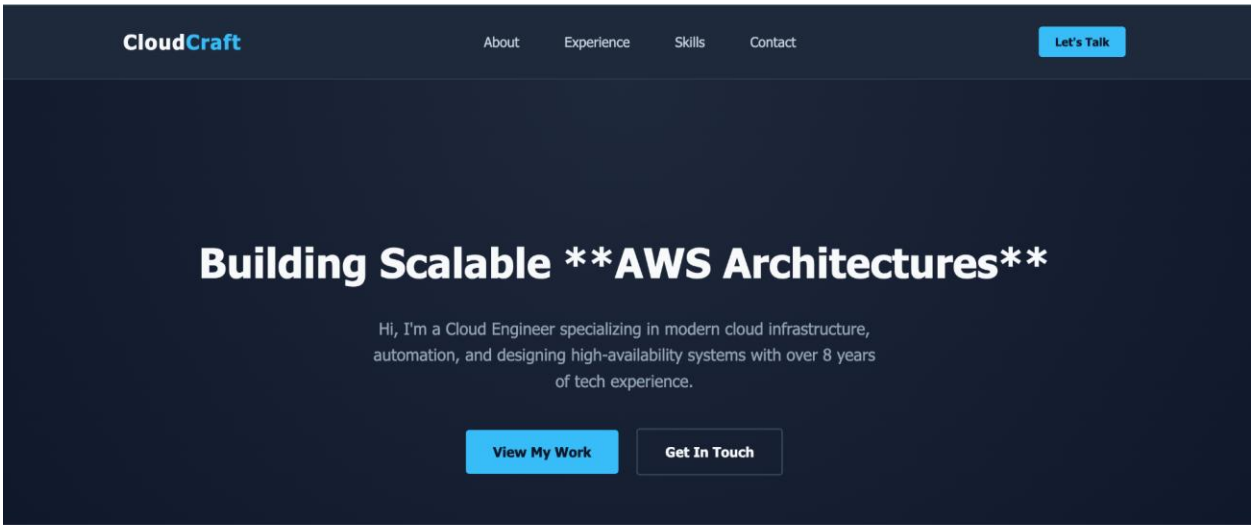
18. Enable static website hosting in the S3 bucket settings and specify the index document.



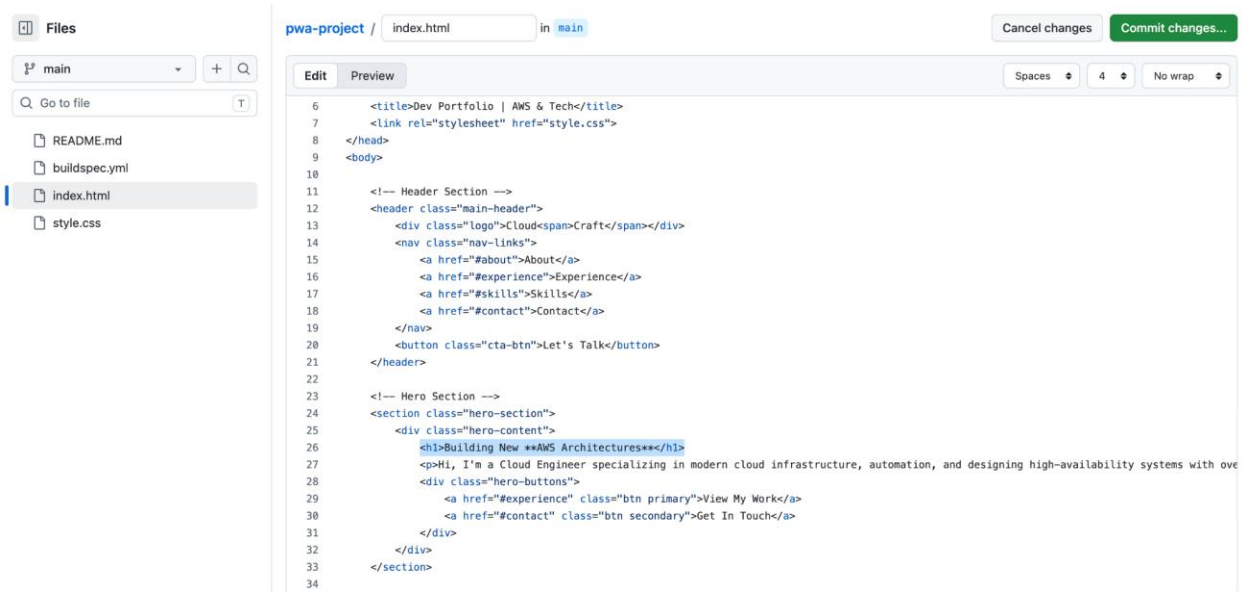
19. Configure public read permissions using an appropriate bucket policy.



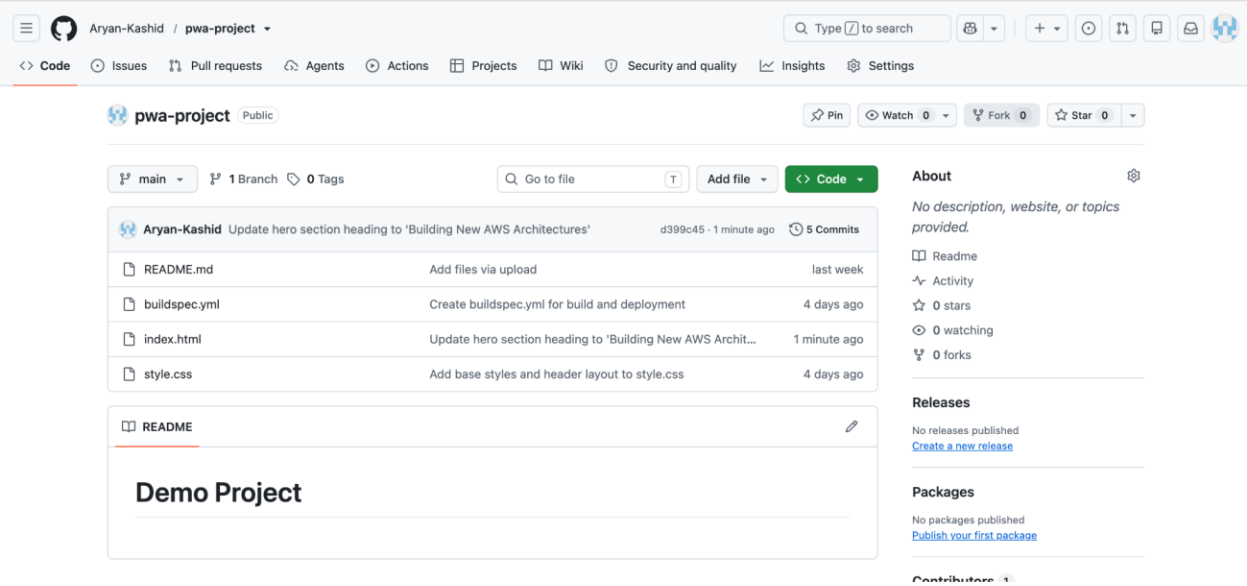
20. Access the static website endpoint to verify initial deployment.



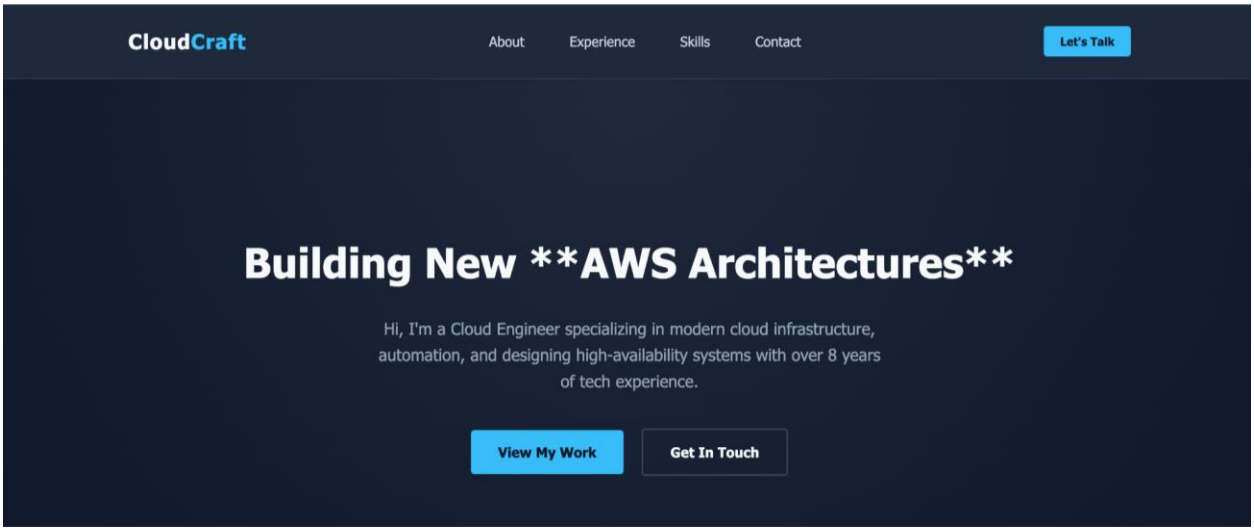
21. Update index.html in GitHub repository to test automatic triggering.



22. Commit and push the changes to trigger the pipeline automatically.



23. Refresh the website endpoint to confirm that updated changes are automatically deployed.



Conclusion :

Thus, we successfully implemented an automated CI/CD pipeline on AWS using GitHub, AWS CodeBuild, AWS CodePipeline, and Amazon S3. The pipeline automatically fetched the source code from GitHub, built the application using CodeBuild, and deployed the website files to an S3 bucket configured for static website hosting. We also verified that changes made to the source code were automatically reflected on the deployed website. Hence, the experiment demonstrated the automation of the end-to-end software build and deployment process using AWS services.